

Exponent Orderbook

Smart Contract Security Assessment

August 2025

Prepared for:

Exponent Finance

Prepared by:

Offside Labs

Zhoujie Tong

Ripples Wen





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Missing Ownership Check When Removing Offers	6
4.2	Incorrect Quote Amount Calculation in Virtual Sell YT Operation	6
4.3	Uses Incorrect Exchange Rate After Vault Expiry When Collect Interest	7
4.4	Missing Fee Handling When Collecting Interest	8
4.5	User Funds Lost When Order Amount Is Too Small to Settle	9
4.6	Inefficient Iteration and Redundant Computation in Offer Matching	10
4.7	Missing User Escrow Cleanup in collect_interest IX	11
4.8	Missing mut Declaration on Admin Account in Lamports Transfer	11
4.9	Missing Expiry Check on Offers	12
4.10	Incomplete Conditional Logic for Minting and Redeeming in Wrapper Functions	13
4.11	User Escrow Remains Allocated When Posting Offer Below Threshold	14
4.12	Omission of Emission Rewards	15
4.13	Informational and Undetermined Issues	16
5	Disclaimer	18



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Exponent* smart contracts, starting on July 07, 2025, and concluding on July 16, 2025.

Project Overview

The Exponent Orderbook is a fully on-chain orderbook built on Solana that enables efficient trading of yield tokens (YT) and principal tokens (PT) against synthetic tokens (SY) without requiring a crank mechanism. It serves as a core component of the Exponent Finance ecosystem, providing a decentralized marketplace for yield token trading with sophisticated order matching and price discovery mechanisms.

The contract maintains a traditional orderbook with price-time priority matching, establishing fair market prices through advanced order matching algorithms. Users can place limit orders, execute market orders, cancel pending orders, and withdraw their tokens and earned interest through both direct and wrapper functions. The system automatically collects and distributes interest from yield positions while implementing comprehensive safeguards to protect users and maintain market stability.

Key innovations include automatic strip/merge operations that seamlessly convert between SY and PT+YT tokens during trade execution, APY-based pricing that naturally reflects yield token economics, and time-decay pricing mechanisms that account for token maturity and remaining time to expiration.

Audit Scope

The assessment scope contains mainly the smart contracts of the Exponent-Orderbook program for the *Exponent* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- *Exponent*
 - Codebase: <https://github.com/exponent-finance/exponent-core>
 - Commit Hash: ed92efb4e93d7c3f07167a4742305f58693408b7

We listed the files we have audited below:

- *Exponent*
 - solana/programs/exponent_orderbook/src/allocator.rs
 - solana/programs/exponent_orderbook/src/lib.rs
 - solana/programs/exponent_orderbook/src/instructions/orderbook/remove_offer.rs
 - solana/programs/exponent_orderbook/src/instructions/orderbook/market_offer.rs
 - solana/programs/exponent_orderbook/src/instructions/orderbook/collect_interest.rs



- solana/programs/exponent_orderbook/src/instructions/orderbook/post_offer.rs
- solana/programs/exponent_orderbook/src/instructions/orderbook/util.rs
- solana/programs/exponent_orderbook/src/instructions/orderbook/admin/realloc_orderbook_account.rs
- solana/programs/exponent_orderbook/src/instructions/orderbook/admin/orderbook_init.rs
- solana/programs/exponent_orderbook/src/instructions/util.rs
- solana/programs/exponent_orderbook/src/utls/core_cpi.rs
- solana/programs/exponent_orderbook/src/utls/pda.rs
- solana/programs/exponent_orderbook/src/state/offer.rs
- solana/programs/exponent_orderbook/src/state/cpi_accounts_orderbook.rs
- solana/programs/exponent_orderbook/src/state/price_node.rs
- solana/programs/exponent_orderbook/src/state/orderbook.rs
- solana/programs/exponent_orderbook/src/state/user_escrow.rs

Findings

The security audit revealed:

- 3 critical issues
- 2 high issues
- 0 medium issue
- 7 low issues
- 3 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Missing Ownership Check When Removing Offers	Critical	Fixed
02	Incorrect Quote Amount Calculation in Virtual Sell YT Operation	Critical	Fixed
03	Uses Incorrect Exchange Rate After Vault Expiry When Collect Interest	Critical	Fixed
04	Missing Fee Handling When Collecting Interest	High	Fixed
05	User Funds Lost When Order Amount Is Too Small to Settle	High	Fixed
06	Inefficient Iteration and Redundant Computation in Offer Matching	Low	Fixed
07	Missing User Escrow Cleanup in collect_interest IX	Low	Fixed
08	Missing mut Declaration on Admin Account in Lamports Transfer	Low	Fixed
09	Missing Expiry Check on Offers	Low	Fixed
10	Incomplete Conditional Logic for Minting and Redeeming in Wrapper Functions	Low	Fixed
11	User Escrow Remains Allocated When Posting Offer Below Threshold	Low	Fixed
12	Omission of Emission Rewards	Low	Fixed
13	Semantic Inconsistency: Misuse of FOK Logic in Offer Matching	Informational	Fixed
14	Unused Accounts Passed into Instructions	Informational	Fixed
15	Inaccurate Handling of Exact Match in FoK Orders	Informational	Fixed



4 Key Findings and Recommendations

4.1 Missing Ownership Check When Removing Offers

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The current implementation does not verify whether the offer being removed actually belongs to the trader initiating the removal. There is no ownership validation before the deletion operation is performed.

Impact

Without proper ownership checks, a malicious trader could exploit this flaw to remove offers created by other users. The system is designed to refund the offer amount upon removal, so the attacker can also illegitimately receive funds from offers they do not own. This can lead to unauthorized fund transfers, financial loss, and a breach of trust in the platform's integrity.

Recommendation

An ownership verification step should be implemented before allowing an offer to be removed. Specifically, the system should retrieve the `user_escrow_idx` associated with the given `offer_idx`, then compare `user_escrow.user` with `trader.key` to ensure that the trader is indeed the owner of the offer. If this check fails, the removal operation must be rejected to prevent unauthorized access.

Mitigation Review Log

Fixed in commit 78ac4e36f7642ca48749ae75c1d3d5281df30a3d.

4.2 Incorrect Quote Amount Calculation in Virtual Sell YT Operation

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Math Error

Description

During the virtual sell YT (Yield Token) operation, the system miscalculates the quote amount. The formula used to compute the quote does not accurately reflect the expected



market behavior or protocol design, leading to incorrect output values under certain input conditions.

```
115     let quote: f64 = if offer_type == OfferType::SellYt {  
116         (amount_float / pt_to_sy_rate) - amount_float  
117     } else {
```

[solana/programs/exponent_orderbook/src/instructions/orderbook/util.rs#L115-L117](https://solana.com/programs/exponent_orderbook/src/instructions/orderbook/util.rs#L115-L117)

Impact

This miscalculation directly affects the correctness of the `can_fill` check, which determines whether an offer can be fulfilled under current market conditions. Furthermore, the incorrect quote value propagates into the computation of `taken_amount` during the offer matching process, potentially resulting in incorrect asset transfers.

Proof of Concept

In the current implementation, the quote amount is computed as:

```
quote_amount = amount_float / pt_to_sy_rate - amount_float
```

However, based on the correct pricing model, the computation should be:

```
quote_amount = amount_float / pt_price_asset - amount_float
```

Recommendation

Review the formula used in the `compute_quote_amount_virtual_sell_yt` function to ensure it aligns with the intended economic model of the protocol.

Mitigation Review Log

Fixed in commit 2fbaf364323fa16d7f3347d282862f56c0d02509.

4.3 Uses Incorrect Exchange Rate After Vault Expiry When Collect Interest

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `collect_interest` IX, the program fetches the current `sy_exchange_rate` to calculate the interest accrued by a user's staked YT amount. This approach works correctly before the vault expires, as SY continues to accrue yield.



However, after the vault has expired, Exponent-core records a fixed `final_sy_exchange_rate`, which represents the final yield state. Despite this, `collect_interest` continues to use the outdated and mutable `sy_exchange_rate` instead of switching to `final_sy_exchange_rate`.

As a result, users who call `collect_interest` after expiry will compute their rewards based on an incorrectly increasing exchange rate, allowing them to withdraw more interest than actually exists in the escrow.

Impact

This flawed calculation causes users to over-withdraw interest after the vault expiry, even though the total yield is fixed. Since the orderbook interest escrow does not increase after expiry, excessive withdrawals by early users deplete the shared pool, leaving insufficient funds for others. This creates a first-come-first-serve race condition and can lead to loss of funds for later users, breaking the expected fairness and correctness of the protocol's yield distribution mechanism.

Recommendation

Update the `collect_interest` IX to check whether the vault has expired, and if so, use `final_sy_exchange_rate` instead of the live `sy_exchange_rate`.

Mitigation Review Log

Fixed in commit 0f98f96d89f64d73b0cdc346e5af4bb35cfc6be9.

4.4 Missing Fee Handling When Collecting Interest

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `collect_interest` instruction, the orderbook program calls a CPI into the core program and passes a specified `sy_amount` to be redeemed as interest. The orderbook program then assumes that this full `sy_amount` will be transferred to the user and proceeds accordingly, treating `sy_amount` as the amount the user should receive.

However, within the core module, only part of the `sy_amount` is actually transferred to the destination SY token account. The core program deducts a protocol fee and only transfers the `user_sy` amount. Since the orderbook program is unaware of this deduction, it behaves as if the full `sy_amount` has been paid out to the user.



Impact

This results in users collecting more interest than they are entitled to, as the system fails to account for the protocol fee deducted within the core module. Over time, this leads to a leakage of funds from the orderbook interest escrow, reducing available interest for other users.

Recommendation

Modify the `collect_interest` logic so that it does not assume the user receives the full `sy_amount` passed to the core module. Instead, calculate or retrieve the actual amount `user_sy` transferred to the user by the core module.

Mitigation Review Log

Fixed in commit `c6681bc47288238e6cf481a488ec0fcb0b372e0c`.

4.5 User Funds Lost When Order Amount Is Too Small to Settle

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

Under certain edge conditions, such as after partial matching leaves a remainder that is too small to process, the system rejects recording the taker's order or preserving the remainder of the maker's order. In these scenarios, the maker's offer is removed from the order book.

However, the user assets associated with the unmatched or unprocessed portion are neither returned to the user nor stored on-chain, effectively causing them to be lost. This results in unrecoverable user funds, even though no error or rejection is surfaced to the user.

Impact

This flaw directly leads to loss of user assets under routine usage conditions. It can be triggered unintentionally through small-value offers or partial matches that fall below operational thresholds. Since the system silently removes orders without refunding or preserving the remaining value, affected users permanently lose tokens they submitted. Repeated abuse of this behavior can erode trust in the protocol and, if exploited systematically, result in financial harm.

Recommendation

To prevent user asset loss in edge cases involving small offer amounts or residual values after partial matching, the protocol should record any remaining assets into the user's



`user_escrow` , regardless of whether the order is ultimately posted or matched. By explicitly recording for the unprocessed portion in `user_escrow` , the system ensures that all user-submitted funds remain recoverable and correctly attributed.

Mitigation Review Log

Fixed in commit 6ad83d951e64a8890c4c90e9921a02e2e0c3aa45.

4.6 Inefficient Iteration and Redundant Computation in Offer Matching

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Optimization

Description

The current offer matching implementation performs a full traversal of all `price_node` structures and their nested offers on each match attempt. For each offer, the system calls `can_fill` , which performs a complex set of checks and calculations, including balance sufficiency, expiry, and pricing conditions.

More critically, when a match is successful and a partial amount is filled, the remaining unmatched amount triggers a new iteration that starts again from the beginning of the order book, rather than continuing from the current match point. This results in repeatedly re-evaluating the same offers and price levels, including those that have already been fully considered or matched in previous cycles of the same instruction execution.

Impact

The repeated traversal and excessive computation during offer matching can easily exhaust compute units, resulting in DoS. As a result, users may be unable to submit or match offers reliably.

Recommendation

To improve offer matching efficiency and eliminate redundant traversal, the system should adopt a structurally optimized order management approach:

1. Maintain separate `price_tree` structures for buy and sell offers. This allows each matching attempt to begin from the most favorable price point, which is highest for buyers and lowest for sellers, instead of iterating through all prices.
2. For market offers, where partial fills may occur, record the last matched `price_idx` and `offer_idx` alongside the `remaining_amount` . This enables the matching process to resume precisely where it left off, avoiding repeated scanning of already processed offers.



Implementing these optimizations will ensure that the offer matching engine operates with minimal overhead and maintains deterministic, efficient behavior as market activity grows.

Mitigation Review Log

Fixed in commit 575bc80585ce99bddd9b4d9f25b6468faf33785.

4.7 Missing User Escrow Cleanup in collect_interest IX

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `collect_interest` instruction, the program allows a user to claim accumulated interest. However, it does not check whether the associated `user_escrow` account has become obsolete after the interest is collected.

Impact

The total number of `user_escrow` accounts is limited by account resources. Failing to remove these escrows when they are no longer needed leads to unnecessary occupation of limited size. Over time, this can exhaust the available capacity, resulting in a DoS condition where new users are unable to create required `user_escrow` or participate in the protocol.

Recommendation

Add logic to the `collect_interest` instruction to check whether the `user_escrow` is eligible for removal. If so, remove the `user_escrow`.

Mitigation Review Log

Fixed in commit 15b4f00bd22a6e61878bf061152db10fc4221d0c.

4.8 Missing mut Declaration on Admin Account in Lamports Transfer

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation



Description

In the current implementation, the program attempts to transfer lamports from the admin account to the `yield_position` account. However, the admin account is not marked as `mut` in the account context, which is required when transferring funds from an account.

```
25 #[account(mut)]
26 pub payer: Signer<'info>,
27
28 pub admin_signer: Signer<'info>,
```

[solana/programs/exponent_orderbook/src/instructions/orderbook/admin/orderbook_init.rs#L25-L28](#)

```
224 let ix = anchor_lang::solana_program::system_instruction::transfer(
225     ctx.accounts.admin_signer.key,
226     ctx.accounts.yield_position.key,
227     lamports_to_transfer,
228 );
```

[solana/programs/exponent_orderbook/src/instructions/orderbook/admin/orderbook_init.rs#L224-L228](#)

Impact

Without marking the admin account as mutable, the transfer instruction will fail during execution. This can block the initialization of yield positions and lead to unexpected transaction reverts. It should be noted that if the admin and payer are the same account, the transfer will succeed as the payer is already designated as mutable.

Recommendation

It is recommended to use the payer account instead of the admin account for rent related transfers. Should the admin account still be required as the source of lamports, it must be annotated as `mut` to prevent potential failures at runtime.

Mitigation Review Log

Fixed in commit 140890d45b25e8df596e582260b7a9cff66f0a83.

4.9 Missing Expiry Check on Offers

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation



Description

The current implementation does not validate whether an 'FOK' offer has expired in function `can_fill`. Offers are expected to have an associated expiration timestamp that limits their validity window, but this constraint is not enforced during this process.

```
100 if self.fill_or_kill == 1 {  
101     return self.get_type() == counter_type  
102         && quoted_amount < available_incoming_amount  
103         && !is_empty;  
104 }
```

[solana/programs/exponent_orderbook/src/state/offer.rs#L100-L104](https://github.com/exponent-io/solana/blob/master/programs/exponent_orderbook/src/state/offer.rs#L100-L104)

Impact

Without proper expiry checks, offers that should no longer be valid may still be matched or executed. This could lead to stale or outdated trades being fulfilled, which may not reflect the current market state or the original intention of the offer creator.

Recommendation

Introduce an explicit expiry validation step prior to offer matching or execution. The system should compare the current timestamp against the offer's expiry field using the condition `self.expiry_at > now`.

Mitigation Review Log

Fixed in commit 1447a727c8233eb4e0a29e164eb00fffdab00016.

4.10 Incomplete Conditional Logic for Minting and Redeeming in Wrapper Functions

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the wrapper functions that decide whether to perform mint or redeem operations based on an offer, the current condition used to determine minting behavior is logically flawed. The implementation uses the following condition:

```
24 && (offer_type == OfferType::BuyYt || (virtual_offer && offer_type ==  
    OfferType::SellYt))
```



[solana/programs/exponent_orderbook/src/instructions/wrappers/wrapper_market_offer.rs#L24-L24](#)

```
29  && (offer_type == OfferType::BuyYt || (virtual_offer && offer_type ==  
    OfferType::SellYt))
```

[solana/programs/exponent_orderbook/src/instructions/wrappers/wrapper_post_offer.rs#L29-L29](#)

```
36  if offer_type == OfferType::BuyYt || (virtual_offer && offer_type ==  
    OfferType::SellYt)
```

[solana/programs/exponent_orderbook/src/instructions/wrappers/wrapper_remove_offer.rs#L36-L36](#)

This condition mistakenly allows all `BuyYt` offers, including virtual ones, to trigger minting. However, minting should only occur for non-virtual `BuyYt` offers, while virtual offers should only trigger logic when their type is `SellYt`.

Impact

This logical error leads to incorrect behavior in wrapper execution. Virtual offers may incorrectly cause a mint operation, violating the intended separation between virtual and non-virtual execution paths.

Recommendation

Replace the current condition with a logically correct expression that fully distinguishes between virtual and non-virtual offer contexts. Specifically:

```
(!virtual_offer && offer_type == OfferType::BuyYt) || (virtual_offer &&  
offer_type == OfferType::SellYt)
```

Mitigation Review Log

Fixed in commit 51d687a800185c5209f8c048bfb427ca97f9547e.

4.11 User Escrow Remains Allocated When Posting Offer Below Threshold

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When a user posts an offer with an amount below the threshold, the offer is ignored by the matching engine. However, prior to this ignorance, the system creates a `user_escrow`



entry. This results in a persistent `user_escrow` record being stored, despite the fact that no effective offer exists.

Impact

Because `user_escrow` entries are created even when the offer amount is below the threshold and the offer is ultimately invalid, malicious users can exploit this behavior to repeatedly submit small offers and fill the system with empty escrows. This leads to uncontrolled growth of on-chain state and can exhaust the limited capacity for storing `user_escrow` records, ultimately preventing legitimate users from interacting with the protocol, which effectively results in a denial-of-service condition.

Recommendation

Add a precondition check to ensure the offer amount exceeds the threshold before creating a `user_escrow`. Only initialize and insert escrow entries when the offer has passed validation and is expected to be usable.

Mitigation Review Log

Fixed in commit 2f85ad6064cbf76f6770f327f1e29351124b8ce5

4.12 Omission of Emission Rewards

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In both the Exponent Core and SY Standard programs, yield calculations include not only accrued interest but also protocol emission rewards. However, when a user places an order via the order book, only the interest component is considered, emission rewards are completely omitted by the current implementation.

Impact

Due to this inconsistency, users who utilize the order book may receive lower overall yields compared to those interacting directly with the Exponent Core or SY Standard programs. This disparity in reward distribution may erode user trust and undermine the perceived fairness and transparency of the protocol.



Recommendation

It is recommended to update the order book implementation to comprehensively account for all yield sources, including both interest and emission rewards, in order to maintain consistency with the Exponent Core and SY Standard program calculations.

Mitigation Review Log

Fixed in commit e019c6e3001d5fbbf0bf25e79cb29f83905981ae.

4.13 Informational and Undetermined Issues

Semantic Inconsistency: Misuse of FOK Logic in Offer Matching

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Design

The current implementation exhibits inconsistent handling of partial and full fills in a way that contradicts the standard FOK semantics. When a trader submits a new offer:

1. The system first attempts to match and fill any portion of the offer that is immediately executable.
2. The unmatched remainder is then stored on the order book.
3. However, in subsequent matching attempts, the logic enforces that the entire remainder must be filled at once, resembling FOK behavior.

This is conceptually inconsistent: FOK requires “immediate full execution or cancellation”, not deferred full execution after partial fill. The protocol appears to mix partial-fillable behavior with FOK-style strictness in a manner that defies established trading semantics.

This logic inconsistency can lead to unexpected user experience and confusion for integrators. Furthermore, users may find it difficult to reason about whether and how their offers will be matched, reducing protocol transparency and reliability.

Fixed in commit 1447a727c8233eb4e0a29e164eb00fffdab00016.

Unused Accounts Passed into Instructions

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In certain instructions, some accounts are unnecessarily required, including:

- `escrow_yt` in `initialize_orderbook`
- `yield_position` in `collect_interest`
- `yield_position` in `withdraw_funds`



Requiring these extraneous accounts not only increases transaction size but also introduces additional complexity and potential surface areas for error or misuse. It is generally recommended to limit account requirements to only those strictly necessary for the operation being performed. Including unused or unnecessary accounts may also confuse integrators and auditors, making the code less maintainable and auditable. It's recommended to remove any accounts that are not explicitly utilized by the program logic. This will enhance both efficiency and security.

Fixed in commit 762d9572f519f48a91722855f53dd5f45e824a83.

Inaccurate Handling of Exact Match in FoK Orders

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

```
99 // if fill or kill, we need to check if the order can be filled
   // completely
100 if self.fill_or_kill == 1 {
101     return self.get_type() == counter_type
102         && quoted_amount < available_incoming_amount
103         && !is_empty;
104 }
```

[solana/programs/exponent_orderbook/src/state/offer.rs#L99-L104](https://github.com/solana/programs/exponent_orderbook/src/state/offer.rs#L99-L104)

When verifying whether a FoK order can be filled, the code checks if `quoted_amount < available_incoming_amount`. However, it is more appropriate and accurate to use `quoted_amount <= available_incoming_amount`. If the available incoming amount exactly matches the quoted amount, the current condition would unnecessarily fail the fill. Using a less-than-or-equal-to comparison ensures that orders meeting the exact amount can also be filled as expected, aligning with typical FoK semantics.

Fixed in commit 1447a727c8233eb4e0a29e164eb00fffdab00016.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

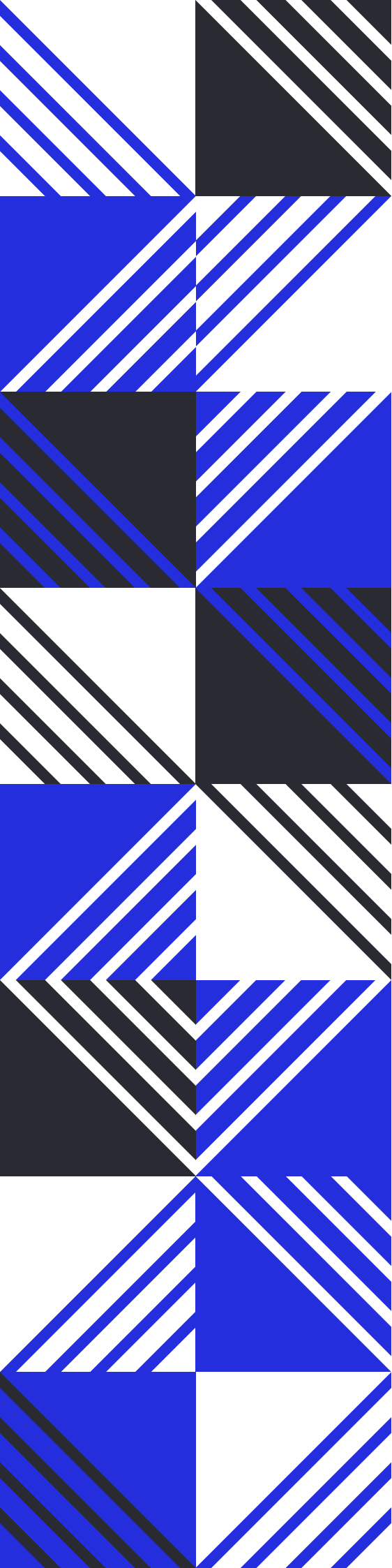
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs